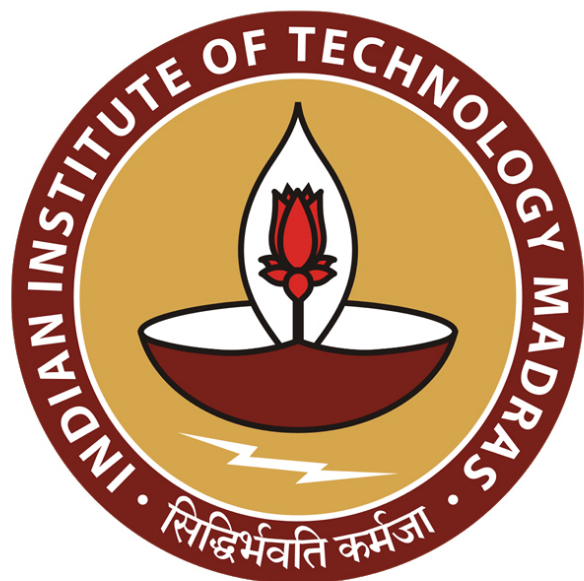


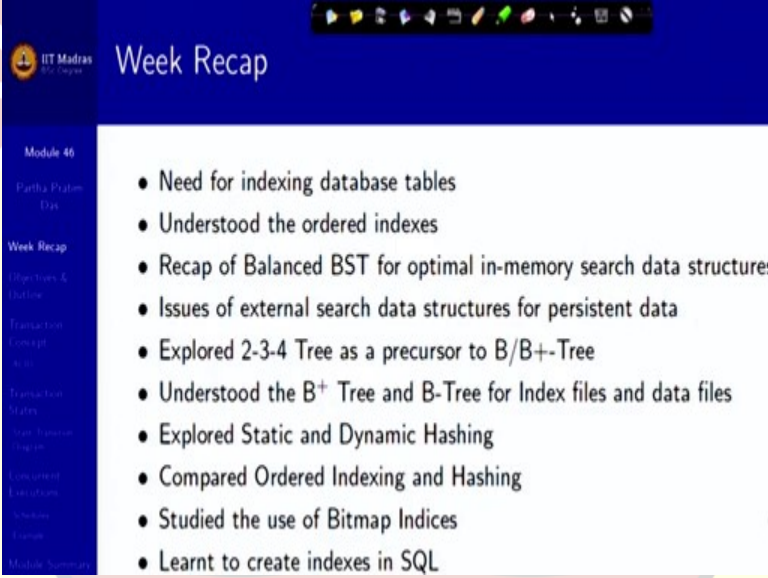


IIT Madras
ONLINE DEGREE

Database Management Systems
Professor. Partha Pratim Das
Department of Computer Science & Engineering
Indian Institute of Technology, Kharagpur
Transactions 1

Welcome to Week 10 Module 46 of Database Management Systems course in the IIT Madras online B.Sc. program.

(Refer Slide Time: 0:31)



The screenshot shows a presentation slide titled "Week Recap" from the IIT Madras online B.Sc. program. The slide is part of Module 46, taught by Professor Partha Pratim Das. The left sidebar lists the following topics: Module 46, Partha Pratim Das, Week Recap, Operations & Offline, Transaction Concept, ACID, Transaction Modes, Query Processor, Consistent Environments, Schedules, Locks, and Module Summary. The main content area lists the following topics covered in the week:

- Need for indexing database tables
- Understood the ordered indexes
- Recap of Balanced BST for optimal in-memory search data structures
- Issues of external search data structures for persistent data
- Explored 2-3-4 Tree as a precursor to B/B+-Tree
- Understood the B⁺ Tree and B-Tree for Index files and data files
- Explored Static and Dynamic Hashing
- Compared Ordered Indexing and Hashing
- Studied the use of Bitmap Indices
- Learnt to create indexes in SQL

The last week was spent on discussing various aspects of data, physical data organization, design considerations, accesses, indexing, hashing, and how to do that, what you should be doing in the real application.

(Refer Slide Time: 0:53)

Module Objectives

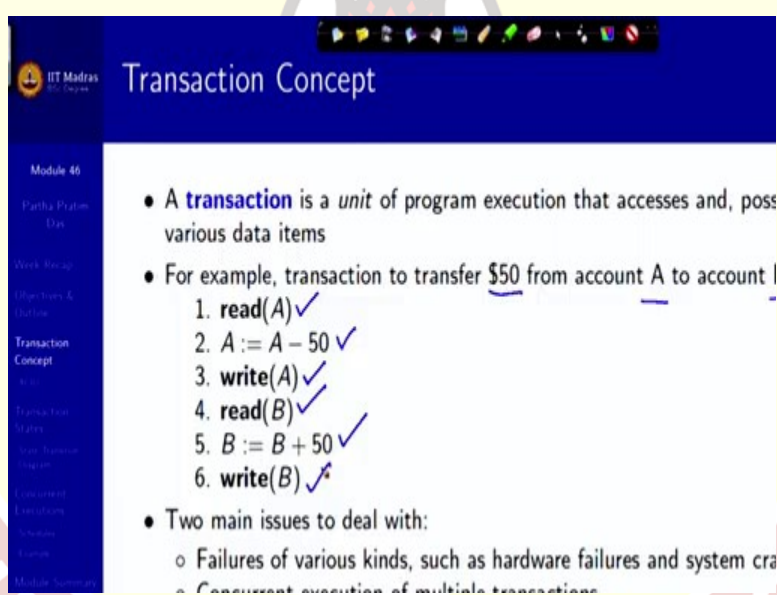
- To understand the concept of transaction – 'doing a task in a databa
- To explore issues in concurrent execution of transactions

Module Outline

- Transaction Concept
- Transaction State
- Concurrent Executions

This week we will introduce and discuss about transactions. So, you had design, you had the runtime data organization, the physical design and now we have the operations to learn about that is a transaction and particularly learn about issues in concurrent transactions.

(Refer Slide Time: 1:14)



Transaction Concept

Module 46
Partha Pratim Das
Work Backup
Objectives & Outline
Transaction Concept
ACID
Transaction States
Lock Protocol
Schedules
Concurrent Execution
Schedules
Example
Module Summary

- A **transaction** is a *unit* of program execution that accesses and, possibly, modifies various data items
- For example, transaction to transfer \$50 from account A to account B:
 1. `read(A)`
 2. `A := A - 50`
 3. `write(A)`
 4. `read(B)`
 5. `B := B + 50`
 6. `write(B)`
- Two main issues to deal with:
 - Failures of various kinds, such as hardware failures and system crashes
 - Concurrent execution of multiple transactions

So, first to define the concept of a, introduce the concept of a transaction. Transaction is a unit of program execution. So, it has a couple of one or more sequence of instructions. And what do they do? They can access that is read, they can update that is write, and in between they can do several other typical computations of programming languages. So, here is a transaction, sample transaction which we will keep on using as a running example for explaining several ideas, transferring \$50 from account A to account B.

So, naturally what you will have to do read A, first know what is the current balance in A, subtract 50. We are making a simplifying assumption here that the account has more than 50 or more dollars available so that this can be done. In reality, there will be condition check also to see that there is enough balance. Otherwise, we will have exceptions thrown on that. So, you deduct the value. So, A has now \$50 less, so you write that value back. So, A's account has been debited by \$50.

Then you read B and we will have to credit that amount to B. So, you take B, add 50 and that is a new value of B you write it back. So, with this what will happen, account A has been debited by \$50, account B has been credited by \$50 and the transaction has taken place. So, this is the basic idea of the transaction.

Now, we will have to remember that failures of various kinds that could be a hardware failure, software failure, system crash and so on, so the transaction, and these things can happen at any point of time. So, what happens to the transaction when such failures happen? And second how

to handle the concurrent execution of multiple transactions that how to allow two or more transactions work with the same data at the same time, in the concurrent time in a certain, with certain restrictions and collaborations to have better system performance.

(Refer Slide Time: 3:46)

Required Properties of a Transaction: ACID: Atomicity

Module 46
Partha Pratim Das
Week Recap
Objectives & Outline
Transaction Concept
ACID
Transaction States
Serial Transaction Schedules
Concurrent Executions
Schedules
Example
Module Summary

- **Atomicity Requirement**
 - If the transaction fails after step 3 and before step 6, money will be "lost" leading to an inconsistent database state
 - ▷ Failure could be due to software or hardware
 - The system should ensure that updates of a partially executed transaction are not reflected in the database

Transaction to transfer money from account A to account B

1. **read(A)**
2. $A := A - 50$
3. **write(A)**
4. **read(B)**
5. $B := B + 50$
6. **write(B)**

Required Properties of a Transaction: ACID: Atomicity

Module 46
Partha Pratim Das
Week Recap
Objectives & Outline
Transaction Concept
ACID
Transaction States
Serial Transaction Schedules
Concurrent Executions
Schedules
Example
Module Summary

- **Atomicity Requirement**
 - If the transaction fails after step 3 and before step 6, money will be "lost" leading to an inconsistent database state
 - ▷ Failure could be due to software or hardware
 - The system should ensure that updates of a partially executed transaction are not reflected in the database

Transaction to transfer money from account A to account B

1. **read(A)**
2. $A := A - 50$
3. **write(A)**
4. **read(B)**
5. $B := B + 50$
6. **write(B)**

Module 46
Partha Pratim Das
Work Backup
Objectives & Outline
Transaction Concept
ACID
Transaction States
Two-Phase Locking
Locking Protocols
Schedules
Concurrency Control
Module Summary

Atomicity Requirement

- If the transaction fails after step 3 and before step 6, money will be "lost" leading to an inconsistent database state
 - ▷ Failure could be due to software or hardware
- The system should ensure that updates of a partially executed transaction are not reflected in the database

Transaction to transfer money from account A to account B

1. **read(A)**
2. $A := A - 50$
3. **write(A)**
4. **read(B)**
5. $B := B + 50$
6. **write(B)**

So, there are certain properties of a transaction that those are acronymed as ACID, atomicity, consistency, isolation and durability. ACID is a very, very basic concept in transactions, and very important, and like any interviewer usually has a propensity to ask questions around ACID. So, be on top of it. So, the first thing is atomicity. In that, that when I do this transaction, suppose I have, I can have failure at any time. So, suppose this write has been done, up to this point it has happened.

Now, the system fails at this point or at this point. If it does, then you have an inconsistency in the data because A has been debited by \$50 and B has not been credited by \$50 yet. So, if it happens at any of these points, if the failure happens at any of these points, then \$50 simply vanishes from the system. So, you have to ensure that updates of partially executed transactions are not finally reflected in the database.

So, we say this is atomicity which means that either the all of the changes go on correctly, completed and reflected in the database or nothing actually happens, so that the state of the database here and the state of the database here would remain same if you did not do anything. So, atomicity is looking at as if this is an atomic operation. So, if you recall your operating system, you will remember that we had atomic operations there to make sure that critical data, critical sections are either executed or not executed at all. Partial execution is not allowed.

(Refer Slide Time: 5:56)

Required Properties of a Transaction: ACID: Consistency Requirement

- **Consistency Requirement**
 - $A + B$ must be unchanged by the execution of the transaction
 - In general, consistency requirements include
 - ▷ Explicitly specified integrity constraints
 - primary keys and foreign keys
 - ▷ Implicit integrity constraints
 - sum of balances of all accounts, minus sum of loan amounts must equal value of cash-in-hand
 - A transaction, when starting to execute, must see a consistent database
 - During transaction execution the database may be temporarily inconsistent

Transaction Log:

Tran	Amount	Account
1. r	\$50	acc
2. A		
3. w		
4. r		
5. B		
6. w		

Required Properties of a Transaction: ACID: Consistency Requirement

- **Consistency Requirement**
 - $A + B$ must be unchanged by the execution of the transaction
 - In general, consistency requirements include
 - ▷ Explicitly specified integrity constraints ✓
 - primary keys and foreign keys
 - ▷ Implicit integrity constraints ✓
 - sum of balances of all accounts, minus sum of loan amounts must equal value of cash-in-hand
 - A transaction, when starting to execute, must see a consistent database
 - During transaction execution the database may be temporarily inconsistent

Transaction Log:

Tran	Amount	Account
1. r	\$50	acc
2. A		
3. w		
4. r		
5. B		
6. w		

Consistency talks about actually having the database in a consistent state. So, what is the consistency in this problem? \$50 is being transferred from account A to account B. A is debited, B is credited. So, before the transaction the value of $A+B$ that you had must be the same after the transaction as well. The value of $A+B$ here and the value of $A+B$ here must be same. So, this is an invariant. So, that cannot change. If it changes then the database is not consistent.

So, there could be the consistency requirements could come from specified integrity constraints, primary key, foreign keys, they try to guarantee consistency or they could be implicit like the sum of balance of all accounts minus the sum of loan accounts must equal cash in hand whatever

you have in accounts, whatever loans you have taken and I mean you take out the loan liability. So, this is some of balances is your asset, loan is your liability. If you subtract what you should get is the cash that you had, have in hand. If this gets violated, you have a consistency problem.

So, a transaction when it starts executing, it says a consistent database. During execution there may be some inconsistency is created we will see. But at the end it has to be again back to a consistent state. So, that is the second property.

(Refer Slide Time: 7:41)

The slide is titled "Required Properties of a Transaction: ACID: Isolation". It contains a section "Isolation Requirement" with a bullet point: "If between steps 3 and 6 (of the fund transfer transaction), another transaction is allowed to access the partially updated database, it will see an inconsistent database (the sum $A + B$ will be less than it should be)".

Below this, a diagram shows two transactions, T1 and T2, with their operations:

T1	T2
1. read(A)	
2. $A := A - 50$	
3. write(A)	
4. read(B)	read(A), read(B), print(A + B)
5. $B := B + 50$	
6. write(B)	

Handwritten annotations on the slide include checkmarks next to T1's steps 1, 2, and 3, and a circle around T2's operations. An arrow points from T2's operations to the text "the sum $A + B$ will be less than it should be".

The third is isolation. Isolation talks about, if more than one transactions work on the same data, it might guarantee that no inconsistent database update is made available. So, what you can look at here is this there is a transaction T1 which is what we have been dealing with, there is another transaction T2 what it does it reads, A reads B and prints $A+B$.

Now, what if I do them concurrently so that first these three are executed in transaction T1, so what has happened A has got debited by 50 and then this gets executed. What will happen? It will read an A now which is less by 50. B has not changed. So, when it prints $A+B$ it will print a value of $A+B$ which is 50 less than what it was when T1 started and then T1 will complete. It will again become consistent. But at this point it will give you a value which is an inconsistent value.

Now, obviously, if you run T1 and T2 serially that is if you run T1 and then you run T2 or if you run T2 then you run T1, then it will always reflect a correct consistent state. So, this is called

isolation that is two transactions working concurrently should not impact each other in a way that database becomes inconsistent. So, the if you run things serially obviously it will be guaranteed, but in general that will not give you the desired throughput.

(Refer Slide Time: 9:48)

Required Properties of a Transaction: ACID: Durabil

Module 46
Partha Pratim Das

Work Book
Objectives & Outline
Transaction Concept
ACID
Transaction States
State Transition Diagram
Consistent Extensions
Schedules
Exercises
Module Summary

- **Durability Requirement**
 - Once the user has been notified that the transaction has completed (that is, the transfer of the \$50 has taken place), the updates to the database by the transaction must persist even if there are software or hardware failures

Transaction to tra count A to accour

1. **read(A)**
2. $A := A - 50$
3. **write(A)**
4. **read(B)**
5. $B := B + 50$
6. **write(B)**

Finally, durability is after the transaction has completed, then the effect of that transaction must be persistent in the database even if further failures happen. After the transaction is successfully completed the database has come to a consistent state and that statement must stay on for any irrespective of any other failures that is called durability.

(Refer Slide Time: 10:16)

ACID Properties

A **transaction** is a unit of program execution that accesses and possibly updates various data items.

- **Atomicity:** Atomicity guarantees that each **transaction is treated as a single unit, completely, or fails completely**
 - If any of the statements constituting a transaction fails to complete, the entire the database is left unchanged
 - Atomicity must be guaranteed in every situation, including power failures, error
- **Consistency:** Consistency ensures that a transaction can only bring the database from one valid state to another, maintaining database invariants
 - Any data written to the database must be valid according to all defined rules, cascades, triggers, and any combination thereof
- **Isolation:** Transactions are often executed concurrently (multiple transactions reach table at the same time)
 - **Isolation ensures that concurrent execution** of transactions leaves the database that would have been obtained if the transactions were executed sequentially

So, this is your ACID. Here I have in one slide summarized the basic atomicity, consistency, isolation and durability properties. This is more for your self study, but is exactly what we have discussed.

(Refer Slide Time: 10:30)

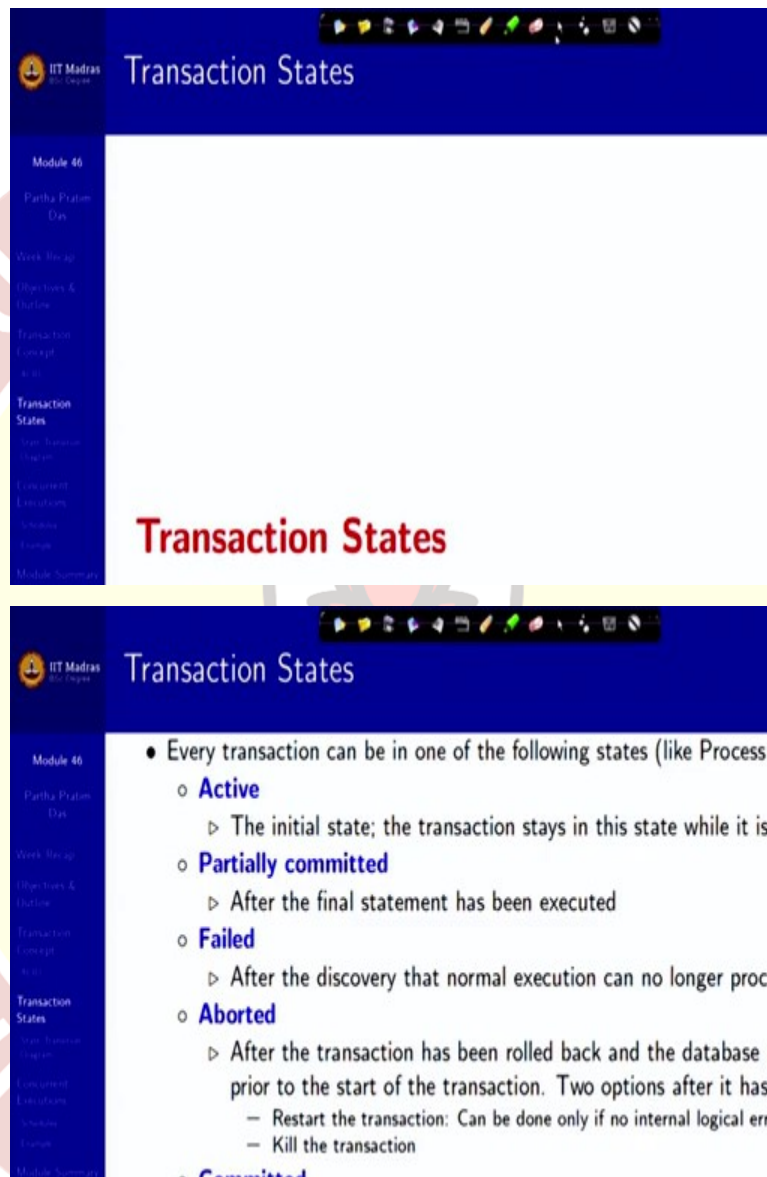
ACID Properties: Quick Reckoner

- A - Atomicity**
All or Nothing Transactions
- C - Consistency**
Guarantees Committed Transaction State
- I - Isolation**
Transactions are Independent
- D - Durability**

And I find this quick reckoner very useful thing is in one slide where in a crisp way atomicity is all or nothing transactions, either you do everything in a transaction or have no effect of the transaction, consistency guarantees committed transaction state, isolation says that transactions

are independent and durability says that committed data is never lost. Get them not only by heart, but by your realization.

(Refer Slide Time: 10:59)



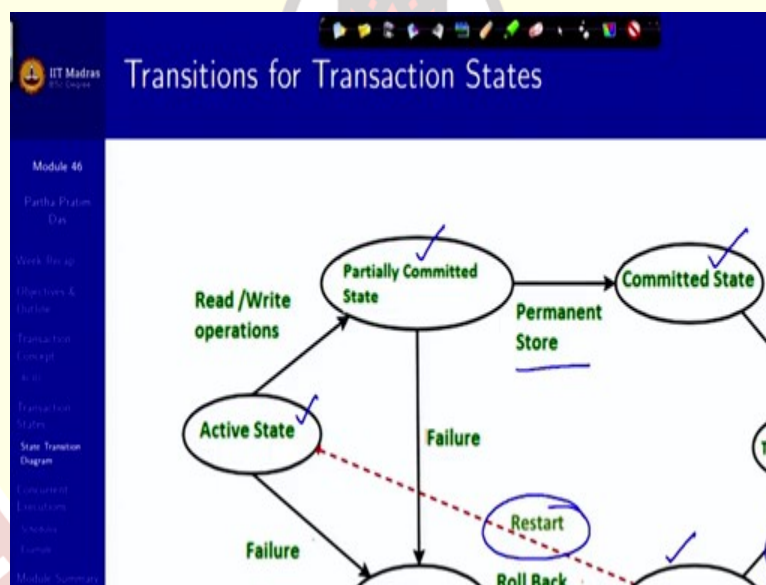
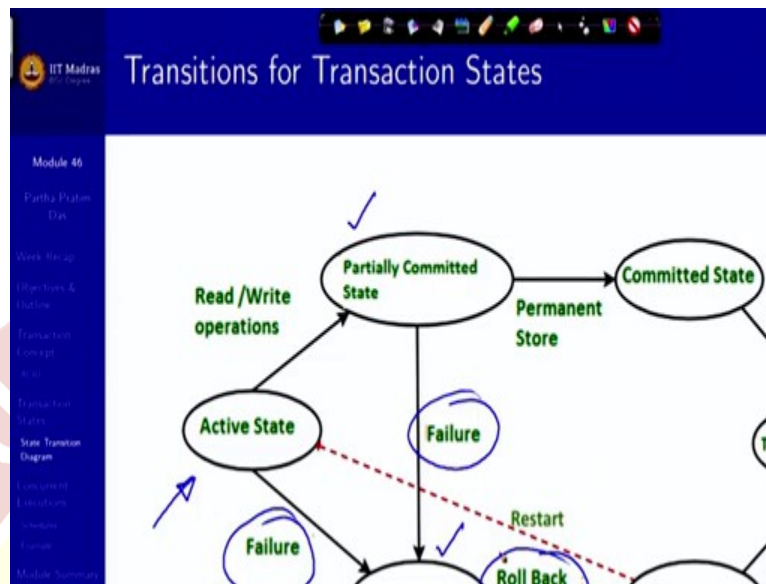
The image displays two screenshots of a presentation slide titled "Transaction States" from IIT Madras. The top screenshot shows the title and a sidebar menu. The bottom screenshot shows a bulleted list of transaction states: Active, Partially committed, Failed, and Aborted, with sub-points for each.

Transaction States

- Every transaction can be in one of the following states (like Process S)
- **Active**
 - ▷ The initial state; the transaction stays in this state while it is
- **Partially committed**
 - ▷ After the final statement has been executed
- **Failed**
 - ▷ After the discovery that normal execution can no longer proceed
- **Aborted**
 - ▷ After the transaction has been rolled back and the database is restored to the state prior to the start of the transaction. Two options after it has
 - Restart the transaction: Can be done only if no internal logical errors occurred
 - Kill the transaction
- **Committed**

Now, transactions are like processes. You have known processes, threads. So, in operating system you have seen that there are states in which a process is, similarly a transaction could be in several different states, which are described here. Let me go to the state transition diagram and explain this.

(Refer Slide Time: 11:22)



So, this is a basic state transition diagram. Initially the transaction is in active state that is when it has begun. So, what are the possibilities? One possibility is it will do some read write operations. And when it has done some read write operations, it comes to a partially committed state. Now, whether it is partially committed because it still has not completed, so it does not have a persistency, does not have a consistent database yet. So, it is just, some commits have happened, others have not, and so on.

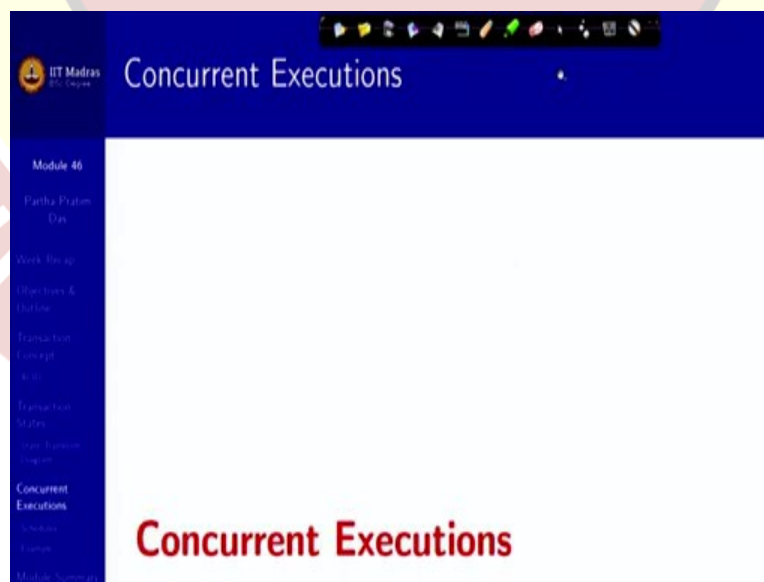
So, if you have a failure while you are in active state or when you are in partially committed state, you go to a new failed state. What you will have to do in the failed state, certain things

which you have already done, you have to undo them, that is rollback, the transaction rollback. This much has been done, this has to be undone. Those, read A assigned A=50, write A then things are failed. So, you have to roll back that write to get to the original value. So, as you rollback you come to a state which is called the aborted state, because your transaction could not go through.

Now, there are two possibilities, one is the system can decide that this was a software error and particularly for software error, logical error and correct for that and it could restart. If it restarts it is back to the active state. The other possibility is a failure actually is some hardware or system failure in which case you cannot actually proceed. So, you have to kill the transaction and you come to the terminated state.

On the success side from the partially committed state if you did not have a failure, then you write back to the permanent store and the transaction goes to a committed state and you are done to come to a terminated state. So, this is the life cycle of the transaction different states and their transition. This is just logically obvious, but you have to learn this diagram, the transition diagram and the process of steps in that by realizing them. So, make use of the earlier slide for descriptions.

(Refer Slide Time: 13:50)



Concurrent Executions

- Multiple transactions are allowed to run concurrently in the system. /
 - **Increased processor and disk utilization**, leading to better trans
 - ▷ For example, one transaction can be using the CPU while another is reading from or writing to the disk
 - **Reduced average response time** for transactions: short transactions wait behind long ones
- **Concurrency Control Schemes:** Mechanisms to achieve isolation
 - To control the interaction among the concurrent transactions in order to prevent them from destroying the consistency of the database

Now, we will talk about concurrent execution which is the most important thing that if we had done all transactions one after the other then the throughput will be very less because we need to increase the processor and disk utilization. For example, while one transaction is using the CPU, the other transaction can do some disk read write. These are independent. So, they can be concurrently done.

Also, you can reduce the average response time because a short transaction which has fired later on, while a long transaction is going on could be done ahead of the long transaction so that effectively you have a better throughput. So, these are the reasons you want to have concurrent execution and for that you need a schedule.

(Refer Slide Time: 14:39)

Schedules

- **Schedule:** A sequence of instructions that specify the chronological order of concurrent transactions are executed
 - A schedule for a set of transactions must consist of all instructions of all transactions
 - Must preserve the order in which the instructions appear in each transaction
- A transaction that successfully completes its execution will have a **commit** as the last statement
 - By default transaction assumed to execute commit instruction as the last statement
- A transaction that fails to successfully complete its execution will have an **abort** as the last statement

The schedule is basically the order in which the instructions are done. Schedule is a sequence of instructions. And those instructions could come from different transactions. So, there are two basic properties of a schedule that it must consist of all instructions of all transactions involved, like you cannot miss out an instruction.

Second, it must preserve the order, which means that in a particular transaction the order in which the instructions are given, you cannot change that in the schedule. You can only do, but you cannot change the order of instructions within one transaction. It must remain the same. Transaction that successfully completes its execution will have the commit as the last statement or commit by default, and which fail will have abort as the last statement.

(Refer Slide Time: 15:41)

Schedule 1

Module 46
Partha Pratim Das
Week Recap
Objectives & Outline
Transaction Concept
ACID
Transaction States
Serial, Parallel, Serializable
Consistent Executions
Schedules
Example
Module Summary

- Let T_1 transfer \$50 from A to B, and T_2 transfer 10% of the balance
- An example of a **serial** schedule in which T_1 is followed by T_2 :

T ₁	T ₂
read (A)	
A := A - 50	
write (A)	
read (B)	
B := B + 50	
write (B)	
commit	
	read (A)
	temp := A * 0.1
	A := A - temp
	write (A)

A	B	A+B	Transactio
100	200	300	@ Start
50	200	250	T1, write A
50	250	300	T1, write B
45	250	295	T2, write A
45	255	300	T2, write B

Consistent @ Cor

Inconsistent @ Tr

So, there is two transactions being done. Transaction T1 transfers \$50 as we have seen and transaction T2 transfer 10% of the balance from A to B. And you can see this is a color code. This is the value of A, this is the value of B, this is the value of A+B which has to be an invariant. And what happens when you do the transactions in different times. And we use the color codes to show the times when it is, the database is consistent. When it is temporarily inconsistent, inconsistent in transit and when it is inconsistent with commit. So, here you are first doing T1 and then doing T2.

So, you have start with a consistent state, deduct \$50 from A and write back. So, as T1 writes A then you are into a inconsistent state because you see that the total has become 250, so \$50 are vanished, but this is in transit. This is just temporary. Then you have the write B of T1 happening here when the added \$50 are written back to B's account and you are back to the consistent state. T1 has finished and committed.

Similar things happen here. You read A and reduce it by 10% 45 temporary inconsistency in transit write back of B it becomes consistent. So, the sequential or the serial schedule will always be consistent.

(Refer Slide Time: 17:34)

Schedule 2

• A serial schedule in which T_2 is followed by T_1 :

T_1	T_2
read (A)	read (A)
$A := A - 50$	$temp := A * 0.1$
write (A)	$A := A - temp$
read (B)	write (A)
$B := B + 50$	read (B)
write (B)	$B := B + temp$
commit	write (B)
	commit

A	B	A+B	Transaction
100	200	300	@ Start
90	200	290	T2, write A
90	210	300	T2, write B
40	210	250	T1, write A
40	260	300	T1, write B

Consistent @ Cor
Inconsistent @ Tr
Inconsistent @ S

Values of A & B are
Schedule 1 – yet

But you could what if you did T2 first and then T1. If you did T2 first, then you will first deduct, transfer 10% from A's account to B's account and then transfer \$50 from A's account to B's account. So, earlier A was having \$45 and B was having \$255, but now A has \$40 and B has \$ 260 because it is a different serial schedule. This is not an error. In a database concurrent system this is perfect, because the database has never got inconsistent. It does not know which one, which transaction has to be done first.

Every transaction is complete and valid in itself. If we have to enforce that, no this is not possible, then you have to design the transaction in this way, but both of them are inconsistent, are consistent at commit. So, they are valid schedules.

(Refer Slide Time: 18:36)

Schedule 3

• Let T_1 and T_2 be the transactions defined previously. The following is a serial schedule, but it is **equivalent** to Schedule 1

Schedule 3

T_1	T_2
read (A)	
$A := A - 50$	
write (A)	
	read (A)
	$temp := A * 0.1$
	$A := A - temp$
	write (A)
	read (B)
	$B := B + 50$
	write (B)
	commit
read (B)	
$B := B + 50$	
write (B)	
commit	

Schedule 1

T_1	T_2
read (A)	
$A := A - 50$	
write (A)	
read (B)	
$B := B + 50$	
write (B)	
commit	
	read (A)
	$temp := A * 0.1$
	$A := A - temp$
	write (A)
	read (B)
	$B := B + temp$

A	B	A+B	Tran
100	200	300	@ SI
50	200	250	T_1, v
45	200	245	T_2, v
45	255	300	T_1, v
45	255	300	T_2, v

Consistent

Inconsistent

Inconsistent

Let us do a third schedule which is not actually serial. So, what we do in the third schedule is transaction T_1 first reads and deducts A. We get here. Then transaction T_2 reads that A and writes it back. You are here. Then transaction T_1 reads B and updates B. Now, what has happened? Transaction 1 has finished. So, it is committed. So, you have a commit.

But still \$5 are not there that you took away in this temp. They have not come into the. So, you reach a state which is inconsistent at commit is as different from these two, because till then no commit had happened. It was inconsistent in transit. But this one is inconsistent at commit. So, this is 295 is inconsistent at commit. Fortunately, as you complete transaction B which reads and adds back this value, I mean, this \$5 from the temp back to the B's account makes it consistent. It is, it gives you the same result as scheduled 1 which is shown side by side.

So, it is possible that during when you have a schedule running concurrent transactions, then it is possible that temporarily transaction, the database could be an inconsistent state even it could be in inconsistent state after commit as long as you do not have an inconsistent state at the end of both transactions, you are okay with that, because in all of these schedules 1, 2 and 3, the sum $A+B$ is preserved to be 300. So, this kind of a schedule 3 is called equivalent to schedule 1 because it will have the same effect in the database.

(Refer Slide Time: 20:53)

Schedule 4

- The following concurrent schedule does not preserve the sum of "A + B"

	T ₁	T ₂
read (A)	✓	✓
A := A - 50	✓	
temp := A * 0.1		✓
A := A - temp		✓
write (A)	✓	✓
read (B)	✓	✓
B := B + 50	✓	
write (B)	✓	
commit	✓	
B := B + temp		✓

Handwritten notes:

- For T₁: A = 50, B = 200, A+B = 250
- For T₂: A = 100, temp = 10, A = 90, B = 200, B = 210

A	B	A+B	Transaction
100	200	300	@ Start
90	200	290	T ₂ , write A
50	200	250	T ₁ , write A
50	250	300	T ₁ , write B
50	210	260	T ₂ , write B

Consistent @ Cor
Inconsistent @ Tr
Inconsistent @ Co

So, we have seen schedules 1, 2 and 3 which are serializable in that they are equivalent to, these concurrent schedules are equivalent to some serial schedule. Now, we will see an example in schedule 4 where the concurrency schedule is not serializable that is it is not, cannot be equivalent to any other serial schedule that is very precisely it will not preserve the sum $A+B$. This invariance will not be preserved.

So, you can see that we have the same set of instructions in T₁ and T₂ as before, but they are arranged intermixed in a different way. So, first we do read A and perform decrease A by 50. So, in the local area of T₁ we have A as 50 now when we are here. We have read A which was 100 and in the local it is 50 for the transaction T₁. Now, the next two instruction to execute is read A by T₂. So, transaction T₂ also will read A as 100. It will read A as 100 because it has not been changed, not been written by transaction T₁, so it will still read 100.

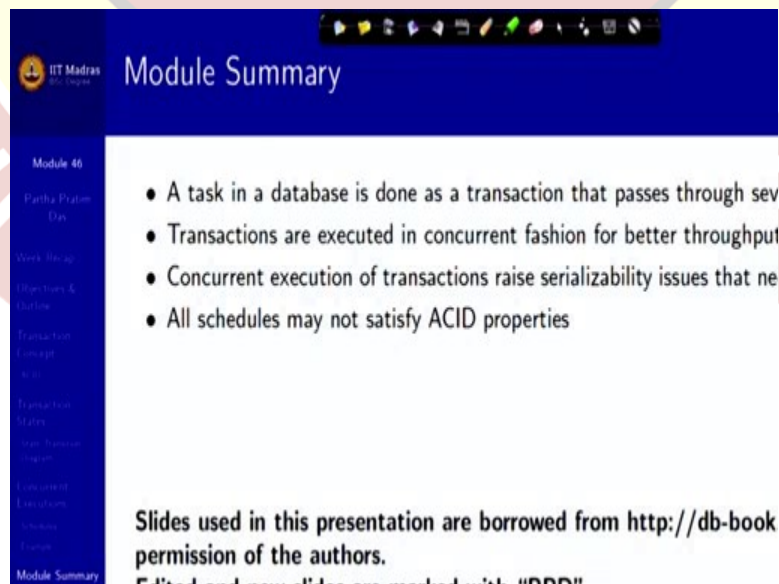
Then temp is so temp in the local of T₂ is 100×0.1 , so temp is 10. So, local in A now when you subtract temp from A becomes 90. So, you will write that A. So, transaction T₂ writes that A. So, this 90 gets written and after this write has been done, you are in a inconsistent, transient inconsistent state because they do not add up to 300 anymore as they should have. And remember that the local copy of A in T₁ remains to be 50. Then you read B so you read 200 and this is what we have in the local copy of T₂.

The next instruction is writing A by transaction T1. So, it writes A. What does it write? It writes whatever is the value of its local copy which is 50. So, it writes 50. So, after T1 has written, the database remains to be in a transient inconsistent state. Then it reads B. What is B? B is 200. So, it read B. B is 200 in the local copy of transaction T1, increments B so it becomes 250. In the local copy, it writes B. So, it writes B now. This is where it is written 250 and it does a commit. As it does the commit. A is 50 written earlier here and B is 250 written here. So, right now the transaction is in a consistent state after this commit.

Now, this instruction is executed. So, B gets $B + \text{temp}$. So, B becomes 210. And you write B. And this is where the problem happens because the old value of A that was written was not the modified value of A that you had computed here because that was overwritten and based on that B is getting changed and so the earlier value of 250 gets changed. So, it is now written as 210 which is the value of the local copy of B in the transaction T2 consequently and then when the transaction commits consequently you have a inconsistent, final inconsistent committed state.

So, this shows that schedule 4 does not preserve the sum. The sum has now become 260, INR40 has simply disappeared. And therefore, there is no serial schedule equivalent to this schedule 4 and it is not serializable. So, that is the point that we make that all concurrent schedules are not serializable.

(Refer Slide Time: 25:45)



Module Summary

- A task in a database is done as a transaction that passes through sev
- Transactions are executed in concurrent fashion for better throughput
- Concurrent execution of transactions raise serializability issues that ne
- All schedules may not satisfy ACID properties

Slides used in this presentation are borrowed from <http://db-book> permission of the authors.

So, this brings us to the end of this module. We have taken a look at a transaction as a task execution in the database. We have talked about the basic concept, the concurrency, the states of a transaction, the concurrency requirements and the concept of schedule and how schedules, concurrent schedules may or may not be equivalent to a serial schedule. Thank you very much for your attention and we will meet and continue on this in the next module.

